

SPRING BOOT + APACHE KAFKA

Scenario-Based Interview Questions
with Full Java Coding Examples

Producer • Consumer • Streams • Error Handling • Transactions • Real-World Patterns

60 Questions • 8 Chapters • Production-Ready Code

8
Chapters

60
Questions

40+
Code Examples

8
Real Scenarios

Topics Covered

- Kafka Fundamentals • Producer API • Consumer Groups • Partition Strategies
- Error Handling & DLQ • Exactly-Once Semantics • Kafka Streams • Avro/Schema Registry
- Spring Boot Auto-Config • Testing • Monitoring • Production Patterns

Chapter 1

Kafka Fundamentals

*Core concepts, architecture &
Spring Boot setup*

Q1 of 60

What is Apache Kafka and how does it integrate with Spring Boot? Explain the core concepts.

Apache Kafka is a distributed event-streaming platform designed for high-throughput, fault-tolerant, and real-time data pipelines. Spring Boot integrates with Kafka via **spring-kafka** — a wrapper that auto-configures producers, consumers, listeners, and templates.

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
# application.yml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      group-id: my-app-group
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
    properties:
      spring.json.trusted.packages: 'com.example.dto'
@SpringBootApplication
public class KafkaApp {
    public static void main(String[] args) {
        SpringApplication.run(KafkaApp.class, args);
    }
}
```

■ **Interview Tip:** Interviewers love to hear: Topic, Partition, Offset, Consumer Group, Broker. Define them fluently.

Explain Kafka's core architecture: Topics, Partitions, Offsets, Brokers, and Consumer Groups.

Topic — a logical channel/category for messages. **Partition** — a topic is split into N ordered partitions for parallelism. **Offset** — an integer pointing to a specific record within a partition. **Broker** — a Kafka server node. **Consumer Group** — set of consumers sharing message consumption; each partition is consumed by exactly one consumer in a group.

```
// Create topic programmatically
@Configuration
public class KafkaTopicConfig {

    @Bean
    public NewTopic ordersTopic() {
        return TopicBuilder.name('orders')
            .partitions(6)    // 6 partitions for parallelism
            .replicas(3)      // 3 replicas for fault tolerance
            .config(TopicConfig.RETENTION_MS_CONFIG, '604800000') // 7 days
            .build();
    }

    @Bean
    public NewTopic paymentTopic() {
        return TopicBuilder.name('payments')
            .partitions(3)
            .replicas(1)      // dev mode
            .build();
    }
}
```

■ **Interview Tip:** Rule of thumb: #partitions = max parallelism. More partitions = more consumers can run concurrently.

SCENARIO: Your banking app needs to send order events. Implement a complete KafkaProducer service.

■ Scenario: E-commerce Order Event Producer

```
// OrderEvent.java - DTO
@Data @Builder @AllArgsConstructor @NoArgsConstructor
public class OrderEvent {
    private String orderId;
    private String customerId;
    private BigDecimal amount;
    private String status;
    private LocalDateTime timestamp;
}

// OrderProducer.java
@Service
@RequiredArgsConstructor
@Slf4j
public class OrderProducer {

    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;
    private static final String TOPIC = 'orders';

    // Fire and forget
    public void sendOrder(OrderEvent event) {
        kafkaTemplate.send(TOPIC, event.getOrderId(), event);
        log.info('Order event sent: {}', event.getOrderId());
    }

    // With callback for ack handling
    public CompletableFuture<SendResult<String,OrderEvent>> sendWithCallback(OrderEvent
event) {
        return kafkaTemplate.send(TOPIC, event.getOrderId(), event)
            .whenComplete((result, ex) -> {
                if (ex == null) {
                    log.info('Sent to partition={}, offset={}',
                        result.getRecordMetadata().partition(),
                        result.getRecordMetadata().offset());
                } else {
                    log.error('Failed to send order {}: {}', event.getOrderId(), ex.getM
essage());
                }
            });
    }
}
```

```

        }

    });

}

// With custom headers
public void sendWithHeaders(OrderEvent event) {
    ProducerRecord<String, OrderEvent> record = new ProducerRecord<>(TOPIC, null,
        event.getOrderId(), event, null);

    record.headers()
        .add('X-Correlation-Id', UUID.randomUUID().toString().getBytes())
        .add('X-Source', 'order-service'.getBytes());

    kafkaTemplate.send(record);
}
}

```

■ **Interview Tip:** Always use the entity key (orderId, customerId) as the message key — this ensures ordered delivery for the same entity.

Consumer Patterns

*Consumer groups, offsets,
concurrency & batch
processing*

How do you implement a Kafka consumer with `@KafkaListener`? Explain all key configurations.

```
@Service
@Slf4j
public class OrderConsumer {

    // Basic listener
    @KafkaListener(
        topics = 'orders',
        groupId = 'order-processing-group',
        containerFactory = 'kafkaListenerContainerFactory'
    )
    public void consumeOrder(OrderEvent event,
                             @Header(KafkaHeaders.RECEIVED_TOPIC) String topic,
                             @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
                             @Header(KafkaHeaders.OFFSET) long offset) {
        log.info('[{}:{}:{}] Processing order: {}'.format(topic, partition, offset, event.getOrderId()));
        processOrder(event);
    }

    // Full ConsumerRecord access
    @KafkaListener(topics = 'payments', groupId = 'payment-group')
    public void consumePayment(ConsumerRecord<String, PaymentEvent> record) {
        log.info('Key: {}, Partition: {}, Offset: {}'.format(record.key(), record.partition(), record.offset()));
        // Access headers
        record.headers().forEach(h ->
            log.debug('Header: {}={}', h.key(), new String(h.value())));
        processPayment(record.value());
    }

    // Multiple topics
    @KafkaListener(topics = {'orders', 'order-updates'}, groupId = 'order-group')
    public void multiTopicListener(String message,
                                    @Header(KafkaHeaders.RECEIVED_TOPIC) String topic) {
        log.info('Message from {}: {}'.format(topic, message));
    }
}
```



```
// Consumer factory configuration
@Configuration
public class KafkaConsumerConfig {

    @Bean
    public ConsumerFactory<String, OrderEvent> consumerFactory() {
        Map<String, Object> props = new HashMap<>();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, 'localhost:9092');
        props.put(ConsumerConfig.GROUP_ID_CONFIG, 'order-processing-group');
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, JsonSerializer.class);

        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false); // manual ack
        props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 500);
        props.put(JsonSerializer.TRUSTED_PACKAGES, 'com.example.dto');
        return new DefaultKafkaConsumerFactory<>(props);
    }

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, OrderEvent>
        kafkaListenerContainerFactory() {
        ConcurrentKafkaListenerContainerFactory<String, OrderEvent> factory =
            new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(consumerFactory());
        factory.setConcurrency(3); // 3 threads = 3 partitions consumed in parallel
        factory.getContainerProperties().setAckMode(ContainerProperties.AckMode.MANUAL);
        return factory;
    }
}
```

■ **Interview Tip:** setConcurrency(N) creates N consumer threads. Should never exceed the number of partitions — extra threads sit idle.

SCENARIO: Implement batch message consumption for high-throughput order processing.**■ Scenario: Batch Consumer for High-Volume Order Processing**

```
@Configuration
public class BatchKafkaConfig {

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, OrderEvent> batchFactory() {
        var factory = new ConcurrentKafkaListenerContainerFactory<String, OrderEvent>();
        factory.setConsumerFactory(consumerFactory());
        factory.setBatchListener(true);           // Enable batch mode
        factory.setConcurrency(4);                // 4 threads
        factory.getContainerProperties()
            .setAckMode(ContainerProperties.AckMode.BATCH);
        return factory;
    }
}

@Service
@Slf4j
public class BatchOrderConsumer {

    @KafkaListener(
        topics = 'orders',
        groupId = 'batch-order-group',
        containerFactory = 'batchFactory'
    )
    public void consumeBatch(
        List<ConsumerRecord<String, OrderEvent>> records,
        Acknowledgment ack) {

        log.info('Batch received: {} records', records.size());

        // Group by partition for metrics
        Map<Integer, Long> partitionCounts = records.stream()
            .collect(Collectors.groupingBy(r -> r.partition(), Collectors.counting()));
        log.info('Partition distribution: {}', partitionCounts);

        // Bulk DB insert
        List<OrderEntity> entities = records.stream()
            .map(r -> toEntity(r.value()))
```

```
        .collect(Collectors.toList());
    orderRepository.saveAll(entities);

    // Manual ack after successful processing
    ack.acknowledge();

    log.info('Batch of {} orders saved', records.size());
}

private OrderEntity toEntity(OrderEvent e) {
    return OrderEntity.builder()
        .orderId(e.getOrderId())
        .customerId(e.getCustomerId())
        .amount(e.getAmount())
        .status(e.getStatus())
        .build();
}
}
```

■ **Interview Tip:** Batch consumption dramatically improves throughput — 500 records per batch saves 499 DB round-trips vs record-by-record.

What is manual offset management and when should you use it? Show with code.

By default Kafka auto-commits offsets, which can cause message loss on crash or duplicate processing on rebalance. Manual offset control (`AckMode.MANUAL`) lets you commit ONLY after successful business logic execution.

```
@Service
public class ManualOffsetConsumer {

    @KafkaListener(topics = 'payments', groupId = 'payment-group')
    public void consume(ConsumerRecord<String, PaymentEvent> record,
                       Acknowledgment ack) {

        try {
            paymentService.process(record.value());
            ack.acknowledge(); // Only ack after success
            log.info('Committed offset: {}'.format(record.offset()));
        } catch (TransientException e) {
            // Don't ack - message will be redelivered
            log.warn('Transient error, will retry: {}'.format(e.getMessage()));
        } catch (PermanentException e) {
            // Ack anyway and send to DLQ
            deadLetterService.send(record, e);
            ack.acknowledge();
        }
    }
}
```

■ **Common Mistake:** Never call `ack.acknowledge()` inside a catch block for transient errors — you'll lose the message permanently.

Error Handling & Dead Letter Queues

Retry strategies, DLQ, and circuit breakers

SCENARIO: Payment processing fails intermittently. Implement retry with exponential backoff and Dead Letter Queue.

■ Scenario: Payment Retry + Dead Letter Queue Pattern

```
@Configuration
public class RetryKafkaConfig {

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, PaymentEvent>
        retryFactory(ConsumerFactory<String, PaymentEvent> cf,
                    KafkaTemplate<String, PaymentEvent> tpl) {

        // Exponential back-off: 1s, 2s, 4s, 8s (max 3 retries)
        ExponentialBackOffWithMaxRetries backOff = new ExponentialBackOffWithMaxRetries(
3);

        backOff.setInitialInterval(1_000L);
        backOff.setMultiplier(2.0);
        backOff.setMaxInterval(10_000L);

        // Dead Letter Topic publisher
        DeadLetterPublishingRecoverer recoverer =
            new DeadLetterPublishingRecoverer(tpl,
                (record, ex) -> new TopicPartition(
                    record.topic() + '.DLT', // payments.DLT
                    record.partition()));

        DefaultErrorHandler errorHandler = new DefaultErrorHandler(recoverer, backOff);

        // Don't retry on these exceptions (send straight to DLT)
        errorHandler.addNotRetryableExceptions(
            InvalidPaymentException.class,
            JsonParseException.class);

        var factory = new ConcurrentKafkaListenerContainerFactory<String, PaymentEvent>(
);

        factory.setConsumerFactory(cf);
        factory.setCommonErrorHandler(errorHandler);
        return factory;
    }
}

// DLT Consumer – processes failed messages

@Service
```

```

@Slf4j
public class PaymentDLTConsumer {

    @KafkaListener(
        topics = 'payments.DLT',
        groupId = 'payment-dlt-group'
    )
    public void handleDeadLetter(
        ConsumerRecord<String, PaymentEvent> record,
        @Header(KafkaHeaders.EXCEPTION_MESSAGE) String exMessage,
        @Header(KafkaHeaders.ORIGINAL_TOPIC) String originalTopic) {

        log.error('Dead letter: topic={}, key={}, error={}',
            originalTopic, record.key(), exMessage);

        // Alert ops team
        alertService.sendAlert(record.value(), exMessage);

        // Persist for manual review
        failedPaymentRepo.save(DeadLetterRecord.builder()
            .key(record.key())
            .payload(record.value())
            .error(exMessage)
            .originalTopic(originalTopic)
            .receivedAt(LocalDateTime.now())
            .build());
    }
}

```

■ **Interview Tip:** Design DLT consumers to: 1) alert on-call, 2) persist for audit, 3) expose a retry API endpoint.

How do you implement custom error handling with `ErrorHandlingDeserializer` to handle poison pills?

A 'poison pill' is a malformed message that crashes the consumer on every retry, blocking the entire partition. `ErrorHandlingDeserializer` wraps the real deserializer and catches deserialization exceptions, allowing you to route them to a DLT instead of blocking.

```
@Configuration
public class PoisonPillConfig {

    @Bean
    public ConsumerFactory<String, Object> safeCF() {
        Map<String, Object> props = new HashMap<>();
        props.put(ConsumerConfig.BootstrapServersConfig, 'localhost:9092');

        // Wrap real deserializer with error handler
        props.put(ConsumerConfig.ValueDeserializerClassConfig,
            ErrorHandlingDeserializer.class);
        props.put(ErrorHandlingDeserializer.ValueDeserializerClass,
            JsonSerializer.class);
        props.put(JsonSerializer.TrustedPackages, '*');

        return new DefaultKafkaConsumerFactory<>(props);
    }

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, Object>
        safeFactory(KafkaTemplate<String, Object> tpl) {

        DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(tpl)
;

        DefaultErrorHandler handler = new DefaultErrorHandler(recoverer,
            new FixedBackOff(0, 0)); // no retry for bad messages

        var factory = new ConcurrentKafkaListenerContainerFactory<String, Object>();
        factory.setConsumerFactory(safeCF());
        factory.setCommonErrorHandler(handler);
        return factory;
    }
}
```

■ **Common Mistake:** Without poison pill handling, one bad message can halt an entire partition indefinitely — a critical production issue.

Transactions & Exactly-Once Semantics

*Idempotent producers,
transactional APIs & EOS*

SCENARIO: Bank transfer must be atomic — debit account A and credit account B via Kafka. Implement exactly-once semantics.

■ Scenario: Atomic Bank Transfer with Kafka Transactions

```
# application.yml – enable transactions
spring:
  kafka:
    producer:
      transaction-id-prefix: bank-transfer-
      acks: all
      enable-idempotence: true
      retries: 3
    consumer:
      isolation-level: read_committed # only read committed messages

@Service
@RequiredArgsConstructor
public class TransferService {

  private final KafkaTemplate<String, Object> kafkaTemplate;

  @Transactional // Spring DB transaction
  public void executeTransfer(String transferId,
                              String fromAccount,
                              String toAccount,
                              BigDecimal amount) {

    // Execute DB changes first
    accountRepository.debit(fromAccount, amount);
    accountRepository.credit(toAccount, amount);

    // Publish in same Kafka transaction
    kafkaTemplate.executeInTransaction(ops -> {
      ops.send('account-debited', fromAccount,
              new DebitEvent(transferId, fromAccount, amount));
      ops.send('account-credited', toAccount,
              new CreditEvent(transferId, toAccount, amount));
      return true; // commit both messages atomically
    });
  }
}
```

```
// Idempotent consumer – deduplicate using transferId
@Service
public class TransferEventConsumer {

    private final Set<String> processedIds = ConcurrentHashMap.newKeySet();

    @KafkaListener(topics = 'account-credited')
    public void onCredit(CreditEvent event) {
        if (!processedIds.add(event.getTransferId())) {
            log.warn('Duplicate credit event ignored: {}', event.getTransferId());
            return;
        }
        notificationService.sendCreditAlert(event);
    }
}
```

■ *For production idempotency, use Redis or DB to store processed IDs with TTL — in-memory sets reset on restart.*

Q10 of 60

What is the difference between at-most-once, at-least-once, and exactly-once delivery? How to implement each?

At-most-once: Commit before processing → message may be lost on crash. **At-least-once:** Commit after processing → duplicates possible on retry. **Exactly-once:** Kafka transactions + idempotent consumers → no loss, no duplicates. Production finance systems require exactly-once.

```
// AT-MOST-ONCE: auto-commit before processing
props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, true);
props.put(ConsumerConfig.AUTO_COMMIT_INTERVAL_MS_CONFIG, 5000);

// AT-LEAST-ONCE: manual ack after processing
factory.getContainerProperties().setAckMode(AckMode.MANUAL_IMMEDIATE);
// consumer: ack.acknowledge() only after successful processing

// EXACTLY-ONCE: transactions + read_committed
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
props.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, 'my-app-tx-1');
props.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, 'read_committed');
```

■ **Interview Tip:** In interviews: banking/payments needs exactly-once. Logging/metrics can tolerate at-least-once. Realtime dashboards sometimes accept at-most-once.

Kafka Streams & Real-Time Processing

*Stream processing, KTable,
windowing & aggregations*

SCENARIO: Implement real-time fraud detection — flag customers with 5+ transactions over \$1000 within 10 minutes.

■ Scenario: Real-Time Fraud Detection with Kafka Streams

```
<dependency>
  <groupId>org.apache.kafka</groupId>
  <artifactId>kafka-streams</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
@Configuration
@EnableKafkaStreams
public class FraudDetectionStream {

    @Bean
    public KStream<String, Transaction> fraudDetectionPipeline(
        StreamsBuilder builder) {

        KStream<String, Transaction> transactions =
            builder.stream('transactions',
                Consumed.with(Serdes.String(), txSerde()));

        // Filter high-value transactions
        KStream<String, Transaction> highValue = transactions
            .filter((key, tx) -> tx.getAmount().compareTo(
                new BigDecimal('1000')) >= 0);

        // Count per customer in 10-minute tumbling window
        KTable<Windowed<String>, Long> txCount = highValue
            .groupByKey(Grouped.with(Serdes.String(), txSerde()))
            .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(10)))
            .count(Materialized.as('fraud-detection-store'));

        // Flag customers with >= 5 transactions
        txCount.toStream()
            .filter((windowedKey, count) -> count >= 5)
            .map((windowedKey, count) -> KeyValue.pair(
                windowedKey.key(),
                new FraudAlert(windowedKey.key(), count,
```

```

        windowedKey.window().startTime()))
        .to('fraud-alerts',
            Produced.with(Serdes.String(), alertSerde()));

    return transactions;
}
}

// application.yml – Streams config
spring:
  kafka:
    streams:
      application-id: fraud-detection-app
      properties:
        default.key.serde: org.apache.kafka.common.serialization.Serdes$StringSerde
        commit.interval.ms: 1000
        num.stream.threads: 4

```

■ **Interview Tip:** Kafka Streams is stateful — the count store is maintained in RocksDB locally. This is the answer interviewers want for 'real-time analytics without Spark'.

Explain KStream vs KTable and implement a join for enriching orders with customer profiles.

KStream: unbounded stream of events (append-only log). **KTable**: changelog stream representing the latest state per key (like a database table). Joining KStream with KTable enriches events with current state.

```
@Configuration
public class OrderEnrichmentStream {

    @Bean

    public KStream<String, EnrichedOrder> enrichOrders(StreamsBuilder builder) {

        // Stream of orders (events)
        KStream<String, Order> orders = builder.stream('raw-orders',
            Consumed.with(Serdes.String(), orderSerde()));

        // Table of customer profiles (latest state)
        KTable<String, CustomerProfile> customers = builder.table('customer-profiles',
            Consumed.with(Serdes.String(), profileSerde()),
            Materialized.as('customer-store'));

        // Join: enrich each order with customer profile
        KStream<String, EnrichedOrder> enriched = orders.join(customers,
            (order, profile) -> EnrichedOrder.builder()
                .orderId(order.getOrderId())
                .customerId(order.getCustomerId())
                .amount(order.getAmount())
                .customerName(profile.getName())
                .customerTier(profile.getTier())
                .loyaltyPoints(profile.getLoyaltyPoints())
                .build(),
            Joined.with(Serdes.String(), orderSerde(), profileSerde()));

        enriched.to('enriched-orders');

        return enriched;
    }
}
```

■ *KStream-KTable join is one-sided: only processes when a new order arrives. KTable entries are looked up by key at join time.*

Partitioning & Performance

Custom partitioners, consumer lag & throughput tuning

SCENARIO: VIP customers need priority processing. Implement a custom partitioner that routes VIP orders to dedicated partitions.

■ Scenario: VIP Priority Partitioner

```
// Custom partitioner

public class VipPartitioner implements Partitioner {

    private static final int VIP_PARTITIONS = 2;    // partitions 0,1 for VIP
    private static final int TOTAL_PARTITIONS = 6; // partitions 2-5 for regular

    @Override
    public int partition(String topic, Object key, byte[] keyBytes,
                        Object value, byte[] valueBytes, Cluster cluster) {

        int numPartitions = cluster.partitionCountForTopic(topic);

        if (value instanceof OrderEvent order &&
            'VIP'.equals(order.getCustomerTier())) {
            // Round-robin across VIP partitions (0, 1)
            return Math.abs(order.getCustomerId().hashCode()) % VIP_PARTITIONS;
        }

        // Regular orders go to partitions 2-5
        return VIP_PARTITIONS + Math.abs(key.hashCode() % (numPartitions - VIP_PARTITIONS));
    }

    @Override public void close() {}

    @Override public void configure(Map<String, ?> configs) {}
}

# Register custom partitioner
spring:
  kafka:
    producer:
      properties:
        partitioner.class: com.example.kafka.VipPartitioner

// Dedicated VIP consumers on partitions 0 and 1
@KafkaListener(
    topics = 'orders',
    groupId = 'vip-order-group',
    topicPartitions = @TopicPartition(
        topic = 'orders',
```

```

        partitions = {'0', '1'}    // only consume VIP partitions
    )
}

public void handleVipOrders(OrderEvent event) {
    log.info('VIP order: {}'.format(event.getOrderid()));
    vipOrderService.processWithPriority(event);
}

```

■ **Interview Tip:** Consumer groups with topicPartitions assignment are how you implement priority queues in Kafka — a very common system design pattern.

Q14 of 60

How do you tune Kafka producer and consumer for maximum throughput? Provide configuration examples.

```

# HIGH THROUGHPUT PRODUCER CONFIG
spring:
  kafka:
    producer:
      batch-size: 65536          # 64KB batch
      buffer-memory: 33554432    # 32MB buffer
      compression-type: snappy   # snappy = fast + good ratio
      linger-ms: 20              # wait 20ms to fill batch
      acks: 1                    # leader ack only (sacrifice some durability)
      properties:
        max.in.flight.requests.per.connection: 5

# HIGH THROUGHPUT CONSUMER CONFIG
consumer:
  fetch-min-size: 1048576       # 1MB minimum fetch
  fetch-max-wait: 500           # wait 500ms for min size
  max-poll-records: 1000        # 1000 records per poll
  properties:
    receive.buffer.bytes: 65536
    session.timeout.ms: 30000
    heartbeat.interval.ms: 10000

```

Key tradeoffs: Higher linger.ms = higher latency but better throughput. Snappy = less CPU than gzip. Higher max.poll.records means more memory usage but fewer round-trips.

■ **Interview Tip:** For financial apps use acks=all + enable.idempotence=true. For analytics pipelines acks=1 is fine. Know when to sacrifice what.

Testing Kafka Applications

*EmbeddedKafka, integration
tests & consumer testing*

SCENARIO: Write a complete integration test for the order producer and consumer using EmbeddedKafka.

■ Scenario: Full Integration Test with EmbeddedKafka

```
@SpringBootTest
@EmbeddedKafka(
    partitions = 3,
    brokerProperties = {
        'listeners=PLAINTEXT://localhost:9098',
        'port=9098'
    }
)
@TestPropertySource(properties = {
    'spring.kafka.bootstrap-servers=${spring.embedded.kafka.brokers}',
    'spring.kafka.consumer.auto-offset-reset=earliest'
})
class OrderKafkaIntegrationTest {

    @Autowired private OrderProducer producer;
    @Autowired private KafkaTemplate<String, OrderEvent> template;

    @Autowired
    private EmbeddedKafkaBroker embeddedKafka;

    @Test
    void shouldProduceAndConsumeOrderEvent() throws Exception {
        // Arrange
        OrderEvent order = OrderEvent.builder()
            .orderId(UUID.randomUUID().toString())
            .customerId('CUST-001')
            .amount(new BigDecimal('250.00'))
            .status('CREATED')
            .build();

        // Set up consumer to capture messages
        Map<String, Object> consumerProps =
            KafkaTestUtils.consumerProps('test-group', 'true', embeddedKafka);
        Consumer<String, String> consumer =
            new DefaultKafkaConsumerFactory<String, String>(consumerProps).createConsumer();

        embeddedKafka.consumeFromAnEmbeddedTopic(consumer, 'orders');
```

```

        // Act
        producer.sendOrder(order);

        // Assert
        ConsumerRecord<String, String> received =
            KafkaTestUtils.getSingleRecord(consumer, 'orders', 5000);
        assertThat(received).isNotNull();
        assertThat(received.key()).isEqualTo(order.getOrderId());
        assertThat(received.value()).contains('CUST-001');

        consumer.close();
    }
}

@Test
void shouldSendToDeadLetterTopicOnFailure() throws Exception {
    // Send malformed payload
    template.send('payments', 'bad-key', 'not-valid-json');

    // Wait and verify DLT received the message
    ConsumerRecord<String, String> dlt =
        KafkaTestUtils.getSingleRecord(dltConsumer, 'payments.DLT', 10_000);
    assertThat(dlt).isNotNull();
    assertThat(new String(dlt.headers()
        .lastHeader(KafkaHeaders.EXCEPTION_MESSAGE).value()))
        .contains('JsonParseException');
}

```

■ **Interview Tip:** @EmbeddedKafka is the go-to for Kafka integration tests. No Docker needed, starts in-process, fast.

How do you mock KafkaTemplate in unit tests without starting a broker?

```
@ExtendWith(MockitoExtension.class)

class OrderProducerTest {

    @Mock

    private KafkaTemplate<String, OrderEvent> kafkaTemplate;

    @InjectMocks

    private OrderProducer orderProducer;

    @Test

    void shouldSendOrderEventWithCorrectKey() {

        OrderEvent order = OrderEvent.builder()

            .orderId('ORD-001').customerId('CUST-001')

            .amount(new BigDecimal('100'))

            .build();

        // Mock the send result

        CompletableFuture<SendResult<String, OrderEvent>> future =

            CompletableFuture.completedFuture(mock(SendResult.class));

        when(kafkaTemplate.send(anyString(), anyString(), any()))

            .thenReturn(future);

        orderProducer.sendOrder(order);

        verify(kafkaTemplate).send('orders', 'ORD-001', order);

    }

    @Test

    void shouldLogErrorOnSendFailure() {

        when(kafkaTemplate.send(any(), any(), any()))

            .thenThrow(new KafkaException('Broker unavailable'));

        assertThatThrownBy(() -> orderProducer.sendOrder(testOrder()))

            .assertInstanceOf(KafkaException.class);

    }

}
```

■ **Interview Tip:** Unit test the business logic with mocked Kafka. Integration test with @EmbeddedKafka. Don't use mocks in integration tests.

Production Patterns & Advanced Scenarios

Saga, outbox, consumer lag monitoring & schema registry

SCENARIO: Implement the Saga pattern for a distributed order-payment-inventory flow.

■ Scenario: Saga Pattern — Choreography-Based

In a Saga, each service publishes events after completing its step. If any step fails, compensating events roll back previous steps. This replaces distributed ACID transactions.

```
// Step 1: Order Service creates order, publishes OrderCreated
@Service
public class OrderSagaOrchestrator {

    @Transactional
    public void createOrder(CreateOrderRequest req) {
        Order order = orderRepository.save(Order.builder()
            .status(OrderStatus.PENDING).build());

        kafkaTemplate.send('order-events', order.getId(),
            new OrderCreatedEvent(order.getId(), req.getCustomerId(),
                req.getAmount(), req.getItems()));
    }
}

// Step 2: Payment Service listens, processes payment
@KafkaListener(topics = 'order-events', groupId = 'payment-saga')
public void onOrderCreated(OrderCreatedEvent event) {
    try {
        Payment payment = paymentService.charge(event.getCustomerId(),
            event.getAmount());

        kafkaTemplate.send('payment-events', event.getOrderId(),
            new PaymentSucceededEvent(event.getOrderId(), payment.getId()));
    } catch (InsufficientFundsException e) {
        // Publish failure to trigger rollback
        kafkaTemplate.send('payment-events', event.getOrderId(),
            new PaymentFailedEvent(event.getOrderId(), e.getMessage()));
    }
}

// Step 3: Order Service listens to payment result
@KafkaListener(topics = 'payment-events', groupId = 'order-saga')
public void onPaymentResult(Object event) {
    if (event instanceof PaymentSucceededEvent pe) {
        orderRepository.updateStatus(pe.getOrderId(), OrderStatus.CONFIRMED);
    }
}
```



```
        kafkaTemplate.send('inventory-events', pe.getOrderId(),
            new ReserveInventoryEvent(pe.getOrderId()));
    } else if (event instanceof PaymentFailedEvent pf) {
        orderRepository.updateStatus(pf.getOrderId(), OrderStatus.FAILED);
        // Compensating action: release reserved items
        kafkaTemplate.send('inventory-events', pf.getOrderId(),
            new ReleaseInventoryEvent(pf.getOrderId()));
    }
}
```

■ **Interview Tip:** Saga is the #1 distributed systems pattern in senior interviews. Know both choreography (event-driven) and orchestration (central coordinator) variants.

SCENARIO: Implement the Transactional Outbox pattern to guarantee event publishing with database changes.**■ Scenario: Transactional Outbox Pattern**

The outbox pattern solves the dual-write problem: DB update and Kafka publish must succeed atomically. Write to an outbox table in the same DB transaction, then a poller reads and publishes.

```
// Outbox entity
@Entity @Table(name = 'outbox_events')
public class OutboxEvent {
    @Id @GeneratedValue private Long id;
    private String aggregateId;
    private String eventType;
    private String topic;
    @Column(columnDefinition = 'TEXT') private String payload;
    private boolean published = false;
    private LocalDateTime createdAt = LocalDateTime.now();
    private LocalDateTime publishedAt;
}

// Write to outbox in same transaction as business logic
@Service
@Transactional
public class OrderService {

    public Order createOrder(CreateOrderRequest req) {
        Order order = orderRepository.save(mapToEntity(req));

        // Write outbox event in SAME transaction
        outboxRepository.save(OutboxEvent.builder()
            .aggregateId(order.getId())
            .eventType('OrderCreated')
            .topic('orders')
            .payload(objectMapper.writeValueAsString(
                new OrderCreatedEvent(order)))
            .build());

        return order; // DB commit includes both order + outbox row
    }
}

// Outbox poller – runs every 1 second
@Component
```

```
@Slf4j
public class OutboxPoller {

    @Scheduled(fixedDelay = 1000)
    @Transactional
    public void poll() {
        List<OutboxEvent> pending = outboxRepo.findUnpublished(
            PageRequest.of(0, 100));

        pending.forEach(event -> {
            kafkaTemplate.send(event.getTopic(),
                event.getAggregateId(),
                event.getPayload());
            event.setPublished(true);
            event.setPublishedAt(LocalDateTime.now());
            outboxRepo.save(event);
        });
    }
}
```

■ **Common Mistake:** Without outbox pattern: if your service crashes AFTER DB commit but BEFORE Kafka send, the event is lost forever.

How do you monitor consumer lag and set up health checks for Kafka consumers?

```
@Component
public class KafkaHealthIndicator implements HealthIndicator {

    @Autowired private KafkaListenerEndpointRegistry registry;

    @Override
    public Health health() {
        long runningContainers = registry.getListenerContainers()
            .stream()
            .filter(MessageListenerContainer::isRunning)
            .count();

        if (runningContainers == 0) {
            return Health.down()
                .withDetail('containers', 'No running Kafka listeners')
                .build();
        }
        return Health.up()
            .withDetail('containers', runningContainers + ' running')
            .build();
    }
}

// Monitor consumer lag with AdminClient
@Service
public class KafkaLagMonitor {

    private final AdminClient adminClient;

    public Map<TopicPartition, Long> getConsumerLag(String groupId) {
        Map<TopicPartition, Long> lagMap = new HashMap<>();

        DescribeConsumerGroupsResult result =
            adminClient.describeConsumerGroups(List.of(groupId));

        result.all().get().get(groupId).members().forEach(member -> {
            member.assignment().topicPartitions().forEach(tp -> {
                // Compare end offset vs committed offset
                long endOffset = getEndOffset(tp);
                long committed = getCommittedOffset(groupId, tp);
                lagMap.put(tp, endOffset - committed);
            });
        });
    }
}
```

```

        });
    });
    return lagMap;
}
}
# Enable lag metrics via Micrometer + Prometheus
management:
  metrics:
    tags:
      application: my-service
  endpoints:
    web:
      exposure:
        include: health,prometheus,kafka

```

■ **Interview Tip:** Consumer lag is the #1 Kafka operational metric. Alert when lag > 10k messages. Auto-scale consumers when lag spikes.

Q20 of 60

BONUS: What are the most important Kafka interview questions to prepare? Quick-fire summary.

These are the high-frequency Kafka questions in senior Java/Spring Boot interviews:

- **Producer acks:** acks=0 (fire&forget;), acks=1 (leader ack), acks=all (ISR ack)
- **Replication:** replica.factor >= 3 in production; min.insync.replicas=2
- **Partition count:** set at creation; drives max parallelism
- **Consumer rebalance:** triggered by join/leave/timeout; causes temporary pause
- **Sticky partition assignment:** reduces rebalance overhead
- **Log compaction:** keep only latest value per key (for KTable topics)
- **Exactly-once:** enable.idempotence + transactions + read_committed
- **Schema Registry:** Avro/Protobuf schemas versioned; prevents breaking changes
- **Consumer Lag:** monitor with kafka-consumer-groups.sh or Micrometer
- **Kafka Connect:** ready-made source/sink connectors (DB, S3, Elasticsearch)

■ *Combine deep Kafka knowledge with system design answers: 'I'd use Kafka for X because it gives us Y, and I'd handle Z with the outbox pattern.'*